

Hierarchical Soft Actor-Critic for Sparse-Reward Long-Horizon Reinforcement Learning

Zahra Abdalla Elashaal

Z.ELASHAAL@UOT.EDU.LY

*Lab. of Robotics, Informatics and Complex Systems,
National Engineering School of Tunis, University of Tunis El Manar,
Tunis, Tunisia
Faculty of Information Technology, University of Tripoli,
Tripoli, Libya*

Afef Hfaiedh

AFEF.HFAIEDH@ENIT.UTM.TN

*Lab. of Robotics, Informatics and Complex Systems,
National Engineering School of Tunis, University of Tunis El Manar,
Tunis, Tunisia*

Nahla Khraief

NAHLA.KHRAIEF@ENIT.UTM.TN

*Lab. of Robotics, Informatics and Complex Systems,
National Engineering School of Tunis, University of Tunis El Manar,
Tunis, Tunisia*

Issmail Ellabib

I.ELLABIB@UOT.EDU.LY

*Computer Engineering Dept.,
University of Tripoli
Tripoli, Libya*

Giansalvo Cirrincione

GIANSALVO.CIRRINCIONE@U-PICARDIE.FR

*Laboratory of Novel Technologies,
University of Picardie Jules Verne
Amiens, France*

Editor:

Abstract

Exploration in sparse-reward long-horizon tasks poses significant challenges for reinforcement learning. To address these challenges, we propose a two-level Hierarchical Reinforcement Learning (HRL) framework. The first level handles high-level strategic planning, while the low-level uses the continuous-control Soft Actor-Critic (SAC) algorithm, and they utilize entropy-regularized policy optimization. The proposed framework was trained and evaluated using the Search-and-Rescue-2 (SAR-2) dataset. HRL-SAC effectively addresses sparse-reward long-horizon search problems characterized by delayed rewards and continuous control, and its outperforming the flat SAC baseline reinforcement learning in terms of success rates, coverage efficiency, and convergence. These findings indicate that hierarchical entropy-regularized policies are a promising solution to tackle long-horizon sparse-reward reinforcement learning tasks.

Keywords: Hierarchical reinforcement learning, Soft actor-critic, Sparse-reward environments, Long-horizon decision-making, Entropy-regularized control.

Keywords: Hierarchical reinforcement learning, Soft actor-critic, Sparse-reward environments, Long-horizon decision-making, Entropy-regularized control.

1 Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for training autonomous agents to solve complex sequential decision-making tasks through environmental interaction Sutton and Barto (2018). By maximizing long-term cumulative returns, RL agents learn to map environmental observations to optimal actions across diverse domains ranging from game playing to robotics. However, standard deep RL algorithms often struggle in environments characterized by sparse extrinsic rewards and long horizons, where the probability of randomly encountering rewarding states is exponentially low. In such settings, conventional exploration strategies frequently fail to propagate credit back to early decision points, leading to stalled convergence or sub-optimal policies.

Hierarchical Reinforcement Learning (HRL) addresses these limitations by introducing temporal and behavioral abstraction. By decomposing a complex task into a hierarchy of sub-problems, HRL enables agents to reason over extended time scales. Typically, a high-level policy (meta-controller) selects long-term sub-goals, while a low-level policy (worker) executes primitive actions to satisfy these objectives. Despite the theoretical appeal of HRL, combining hierarchical structures with continuous control and off-policy learning remains challenging due to non-stationary transition distributions and unstable gradients.

1.1 Related Works

This section will explore the most relevant studies on Hierarchical Reinforcement Learning (HRL) that focus on Long-Horizon Path Planning and Entropy-Regularized Reinforcement Learning.

Hierarchical reinforcement learning has long been studied as a means of addressing the challenges of long-horizon decision making and sparse rewards by introducing temporal abstraction. Early frameworks, such as the option framework, formalized temporally extended actions and established a theoretical foundation for hierarchical policies Sutton et al. (1999). Subsequent work explored hierarchical value functions and semi-Markov decision processes to improve scalability in complex environments Dietterich (2000). More recent methods utilize deep neural networks to learn hierarchical policies in end-to-end manner, such as Feudal RL, Vezhnevets et al. (2017), and option-critical architectures Bacon et al. (2017) enable joint learning of high-level decision making and low-level control. Although these methods demonstrate improved exploration, they often rely on discrete action spaces or lack stability in continuous control settings. Hierarchical methods have also been applied to sparse-reward domains, where high-level policies guide exploration, while low-level controllers handle execution Kulkarni et al. (2016). However, many existing HRL approaches remain sensitive to reward design or struggle with unstable training dynamics, particularly in continuous action spaces. Recent advances in HRL have demonstrated its effectiveness in addressing sparse rewards and long-horizon decision-making in complex environments. Modern HRL approaches build upon earlier temporal abstraction frameworks while leveraging deep neural networks for scalable policy learning. Methods such as the Hierarchical Actor-Critic (HAC) Levy et al. (2017) and Hierarchical Reinforcement learning with Off-

policy correction (HIRO-A) algorithm Chang et al. (2025) decompose decision-making into high-level goal selection and low-level continuous control, allowing improved exploration and sample efficiency. Other recent work as Yan et al. (2024) addresses the problem of navigation with sparse rewards with Hierarchical RL with Multi-Goal (HRL-MG) but this hierarchical architecture is designed to specify the end goal based on the discrete action space. Hierarchical reinforcement learning has been extensively studied as a framework for introducing temporal abstraction into sequential decision making. Comprehensive surveys summarize a wide range of hierarchical formulations, including options, goal-conditioned policies, and multi-level controllers, as well as their theoretical and empirical properties Al-Emran (2015); Pateria et al. (2021); Hutsebaut-Buysse et al. (2022). These surveys identify long-horizon credit assignment and exploration under sparse rewards as persistent open challenges, particularly in continuous control and off-policy learning settings. Another study, while not HRL, divides the learning challenge into action space learning and policy learning in the context of RL, similar to the LASER method Allshire et al. (2021). A different model focuses on control systems defined in infinite-dimensional function spaces, proposing RL architecture algorithms for optimal policy learning across large agent systems Zhang and Li (2025). Unlike standard RL agents that map states to actions, HRL agents create high-level policies for sub-goals and low-level policies for execution, enhancing exploration and credit assignment.

Hierarchical Reinforcement Learning with Continuous Control, Recent works have explored combining hierarchical structures with continuous control policies. Methods such as Hierarchical Actor-Critic Nachum et al. (2018) demonstrate that decomposing tasks into high-level goals and low-level controllers can significantly improve learning efficiency in long-horizon tasks. These methods show promising results, but often rely on deterministic low-level policies or handcrafted goal representations. Moreover studies have explored HRL in continuous and high-dimensional domains. Liu et al. (2021) demonstrate that hierarchical structures hierarchical deep reinforcement learning with automatic sub-goal identification (HADS) improved performance in sparse-reward environments. Despite these advances, training instability and inefficient exploration remain challenges, particularly when scaling HRL to long-horizon continuous control problems. Other works explored incorporate hierarchical structures with continuous control and off-policy learning, and intrinsic rewards to guide low-level and high-level policies operate. Levy et al. (2017) introduce HAC methods that enable end-to-end training of hierarchical policies in continuous domains. While Shen et al. (2020), demonstrate the smooth regularized reinforcement learning effectively constrains the search space and enforces smoothness in the learned policy. However, in its effectiveness, this method may suffer from instability when scaling to complex environments or when reward signals are highly sparse. In addition, the study Xu and Yang (2023) explores hierarchical reinforcement learning with entropy-regularized objectives, showing promising results in long-horizon tasks. Nevertheless, these approaches often rely on deterministic low-level policies or require complex architectural tuning. More recently, some prior work has explored HRL for long-horizon path planning in complex robotic systems. Distributed hierarchical frameworks have been proposed for micro-assembly tasks, demonstrating improved scalability through task decomposition and coordination between multiple controllers Zhao et al. (2025). Lv et al. (2025) presents hierarchical Twin-delayed deep deterministic (H-TD3) algorithm, a long-horizon path planning with a hierarchical framework

operating on different temporal scales. Similarly, hierarchical approaches have been applied to stratospheric airship navigation to address extended planning horizons and delayed rewards Yang et al. (2025). While these methods highlight the effectiveness of hierarchical decomposition in long-horizon settings, they typically rely on task-specific designs or alternative optimization objectives, and do not explicitly incorporate entropy-regularized off-policy learning. Recent work such as Zeng et al. (2025) proposes a hierarchical framework to improve decision-making efficiency in complex environments. The authors design a structured multi-level architecture that decomposes high-level objectives into low-level executable actions, improving scalability and stability. As HRL agents merely optimize for rewards, the agent tends to search the same space redundantly, which reduces the speed of learning. Watanabe et al. (2022) present an Off-Policy Soft Hierarchical Reinforcement Learning (SHIRO) algorithm that maximizes entropy for efficient exploration. Compared to their approach, our proposed framework integrates entropy-regularized Soft Actor-Critic optimization at both meta and low-level controllers, enabling improved exploration and sample efficiency in autonomous search and rescue tasks.

Actor-Critic and Entropy-Regularized Reinforcement Learning, SAC is an off-policy actor-critic algorithm that optimizes a maximum-entropy objective, encouraging stochastic policies that balance exploration and exploitation. By incorporating an entropy term into the reward function, SAC achieves improved stability and sample efficiency compared to deterministic policy gradient methods and become a widely adopted algorithm for continuous control due to its robustness and sample efficiency. SAC has demonstrated strong performance on continuous control benchmarks and robotic manipulation tasks Haarnoja et al. (2018). However, flat SAC policies often struggle in sparse-reward and long-horizon environments due to limited temporal abstraction and inefficient exploration. Entropy-regularized RL has been shown to significantly improve exploration and stability in continuous control tasks. Although entropy regularization promotes local exploration, it does not fully address the challenge of delayed credit assignment over extended time horizons. Several extensions have attempted to improve SAC’s exploration capabilities through intrinsic motivation or auxiliary objectives Burda et al. (2018). Recent extensions of SAC focus on improving exploration in sparse-reward settings through intrinsic motivation, representation learning, or auxiliary objectives Laskin et al. (2020). However, these approaches typically operate at a single temporal scale and do not explicitly model hierarchical structure and long-horizon credit assignment. SAC formulates policy optimization using a maximum-entropy objective and enables efficient off-policy learning using stochastic policies Hou et al. (2020). Although SAC demonstrates strong performance on a wide range of benchmarks, it operates as a flat policy and does not explicitly address long-horizon decision making or sparse-reward credit assignment through temporal behaviors. A similar challenge in autonomous navigation is addressed by Abdallah et al. (2025), they introduced a Deep RL framework for real-time Unmanned Aerial Vehicle (UAV) trajectory optimization in dynamic and partially observable environments. Their work utilizes the Advantage Actor-Critic (A2C) algorithm to navigate 3D spaces, focusing on robust obstacle avoidance and computational efficiency. Their approach demonstrates high adaptability in stochastic settings, focuses on aerial platforms using A2C, while our work extends these principles by using a Hierarchical Soft Actor-Critic architecture, which handles the sparse-reward and long-horizon nature of Search-and-Rescue missions. Also, to balance between exploration

and exploitation, a historical decision-making regularized maximum entropy (HDMRME) RL algorithm is developed by Dong et al. (2024), but this algorithm did not applied in autonomous driving. Another important categorization is based on the learning objective. Value-based methods, such as Deep Q-Networks (DQN) , aim to estimate value functions (e.g., action-value functions) and derive the policy implicitly by selecting actions that maximize the estimated value Al Homsy et al. (2025). Policy-based methods, such as REINFORCE (Policy Gradient), directly optimize the policy without explicitly learning a value function. Also, actor-critic approaches in Kasaura (2025), including Deep Deterministic Policy Gradient (DDPG) Hfaiedh et al. (2024) and Proximal Policy Optimization (PPO) Schulman et al. (2017), combine both paradigms by using an actor network to represent the policy and a critic network to estimate the value function, thereby improving learning stability and efficiency.

The effective learning of policies in long-horizon, sparse-reward reinforcement learning environments remains a central open problem in machine learning. Standard deep reinforcement learning methods often struggle to propagate delayed rewards over extended temporal horizons, leading to inefficient exploration and unstable convergence. Hierarchical reinforcement learning addresses this challenge by introducing temporal abstraction, enabling agents to reason over multiple time scales.

Our contribution: Unlike prior hierarchical RL approaches that either rely on deterministic low-level policies as HAC by Nachum et al. (2018), discrete action spaces such as HRL-MG by Yan et al. (2024)), or task-specific heuristics, our Hierarchical RL of Soft Actor-Critic (HRL-SAC) framework introduces a novel integration of entropy-regularized Soft Actor-Critic optimization across both hierarchical levels within a continuous control setting. While existing SAC extensions address exploration in flat policies, they hard to handle long-horizon sparse rewards through temporal abstraction; similarly, other HRL methods like SHIRO Watanabe et al. (2022) apply soft hierarchies but lack the dual-level entropy regularization and goal-conditioned SAC that enable our approach’s superior sample efficiency and convergence. This dual-entropy HRL formulation provides the theoretically-grounded solution combining SAC’s maximum-entropy stability with hierarchical credit assignment, without auxiliary rewards or architectural tuning. The proposed approach applies entropy regularization at multiple temporal levels, enabling stable off-policy learning and exploration robustness in sparse-reward long-horizon environments. This formulation preserves the theoretical benefits of SAC while addressing delayed credit assignment through hierarchical decomposition, without relying on auxiliary rewards or task-specific heuristics.

The proposed HRL-SAC agent is illustrated in Figure 1. The overall architecture decomposes decision making into meta (high-level) and low-level control layers interacting with a shared environment. The meta-controller operates at a coarser temporal scale and receives the global state from the environment. It consists of a meta-policy and meta-critic networks, which jointly learn to produce a high-level sub-goal g_t . This sub-goal represents an abstract objective or behavioral directive that guides the lower layer over multiple time steps. The low-level controller operates at a finer temporal resolution and is conditioned on both the current state and the sub-goal g_t generated by the meta-controller. It comprises a low-level policy and low-level critic networks, which produce primitive continuous actions executed directly in the environment. The low-level controller is optimized using the Soft Actor-Critic framework to maximize entropy-regularized returns while satisfying the as-

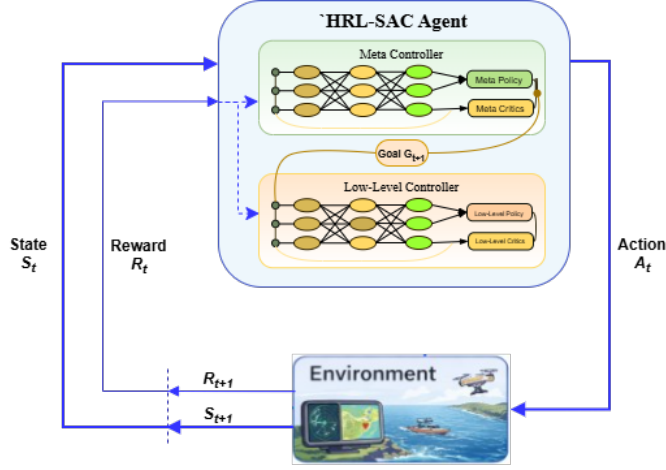


Figure 1: HRL-SAC Agent Architecture. The meta-controller generates high-level sub-goals based on environment observations, while the low-level controller executes entropy-regularized actions conditioned on these sub-goals.

signed sub-goal. The environment provides observations and rewards to both hierarchical levels. Low-level actions influence the environment dynamics, while accumulated task feedback is used to update both the low-level SAC components and the high-level meta-policy. This hierarchical interaction enables efficient learning in long-horizon, sparse-reward tasks by separating strategic planning from reactive control.

2 Theoretical Foundations of Hierarchical Soft Actor-Critic

Reinforcement learning problems are commonly formulated as Markov Decision Processes (MDPs) Sutton and Barto (2018). While modern deep RL methods have achieved impressive results in continuous control, flat policy optimization remains sample-inefficient in sparse-reward and long-horizon environments due to delayed credit assignment. We consider MDP defined by the tuple:

$$M = (S, A, p, r, \gamma),$$

Where S denotes the state space, A the continuous action space, p is the transition kernel, r is the reward function, and $\gamma \in (0, 1)$ the discount factor. In sparse-reward long-horizon environments, direct optimization of flat policies is often inefficient due to delayed credit assignment. HRL addresses this limitation by introducing temporal abstraction, enabling high-level decision making over extended time scales. Separately, entropy-regularized methods such as Soft SAC provide stable and efficient learning by explicitly encouraging stochastic exploration.

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right] \quad (1)$$

where π^* is an optimal policy that maximizes the return, $\mathbb{E}_{\tau \sim \pi}$ is expected value or average future value after the transition, $r(s_t, a_t)$ is a dynamic reward depending on the state s_t and the action a_t at time t , $\mathcal{H}$ is the entropy term, α is the temperature parameter controlling the trade off between maximizing the expected return and promoting exploration through higher policy entropy. SAC optimizes a maximum-entropy objective that increases the expected return with an entropy regularization term, encouraging exploration and robustness Hutsebaut-Buysse et al. (2022). The $Q(s, a)$ denotes the expected long-term return of the current state is expressed as:

$$Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V(s')] \quad (2)$$

The value function $V(s)$ is defined as follows:

$$V(s) = \mathbb{E}_{a \sim \pi}[Q(s, a) - \alpha \log \pi(a | s)] \quad (3)$$

Soft Bellman backup incorporates policy entropy directly into the value estimation, resulting in improved stability in continuous control settings Haarnoja et al. (2018). By combining these two principles into HRL-SAC framework that leverages temporal abstraction and entropy maximization to improve learning efficiency in sparse-reward long-horizon tasks. To address this challenge, in our proposed HRL-SAC framework we decomposed decision-making into two temporal levels: a high-level meta-policy responsible for long-horizon planning, and a low-level Soft Actor-Critic policy responsible for continuous control.

2.1 Hierarchical Policy Structure

The hierarchical policy consists of:

- A (Meta-Agent) uses a stochastic meta-policy $\pi_\phi(g|s)$ that generates a sub-goal $g \in G$ every K time step, ϕ the weights of the high-level policy network, this process handled by the Meta Actors (which likely outputs parameters for a Gaussian distribution).
- a low-level policy $\pi_\theta(a|s, g)$ that produces continuous actions conditioned on the current state and the generated sub-goal g .

At time step t , the execution proceeds as:

$$g_t \sim \pi_\phi(g|s_t) \quad a_{t+i} \sim \pi_\theta(a|s_{t+i}, g_t); \quad i = 0, \dots, K - 1. \quad (4)$$

This temporal abstraction allows the meta-policy to reason over extended horizons, while the low-level policy focuses on short-term control.

2.1.1 LOW-LEVEL SOFT ACTOR-CRITIC OBJECTIVE

The low-level controller is trained using Soft Actor-Critic, which optimizes a maximum entropy objective. The objective for the low-level policy is defined as:

$$J_{LL}(\pi_\theta) = \mathbb{E}_{(s, a, g) \sim \mathcal{B}} [Q_\psi(s, a, g) - \alpha \log \pi_\theta(a | s, g)], \quad (5)$$

where; J_{LL} is the low-level Loss function, Q_ψ denotes the soft Q-function, $\alpha > 0$ is the temperature parameter controlling the entropy-regularization trade-off, and $\mathcal{B}$ is the replay buffer. The corresponding Bellman backup for the soft Q-function is:

$$Q_\psi(s, a, g) = r_{\text{int}}(s, a, g) + \gamma \mathbb{E}_{s' \sim p} [V_{\bar{\psi}}(s', g)], \quad (6)$$

; r_{int} intrinsic reward with the soft value function defined as:

$$V_{\bar{\psi}}(s, g) = \mathbb{E}_{a' \sim \pi_\theta} [Q_{\bar{\psi}}(s', a', g) - \alpha \log \pi_\theta(a' | s', g)]. \quad (7)$$

2.1.2 INTRINSIC REWARD FOR SUB-GOAL EXECUTION

To guide the low-level policy toward accomplishing sub-goals selected by the meta-policy, an intrinsic reward is defined as:

$$r_{\text{int}}(s_t, a_t, g_t) = -\|f(s_{t+1}) - g_t\|_2, \quad (8)$$

where; $\|\cdot\|_2$ is the L_2 norm, $f(\cdot)$ is a mapping function (like a coordinate extractor) that aligns the environment state with the sub-goal space to get goal-relevant representation. This intrinsic reward encourages the low-level controller to minimize the distance between the achieved state and the desired sub-goal, facilitating efficient sub-goal execution and stable learning.

2.1.3 HIGH-LEVEL META-POLICY OPTIMIZATION

The high-level meta-policy operates over a temporally extended MDP, where each action corresponds to selecting a sub-goal g , and transitions occur every K steps. The transition dynamics $p(s'|s, a)$ for the High-Level agent is not standard MDP transitions because they are temporally extended. The transition is defined as (s_t, z_t, R_t, s_{t+K}) , where R_t is the cumulative environment reward or extrinsic return for a meta-action and defined as:

$$R_t^{\text{meta}} = \sum_{i=0}^{K-1} \gamma^i r_{\text{ext}}(s_{t+i}, a_{t+i}), \quad (9)$$

where r_{ext} is the reward for the sparse task, γ is the discount factor between 0-1. The meta-policy is optimized to maximize expected extrinsic return:

$$J_{\text{HL}}(\pi_\phi) = \mathbb{E}_{(s,g) \sim \mathcal{B}_{\text{meta}}} [Q_\omega^{\text{meta}}(s, g)], \quad (10)$$

with the meta-level Bellman update:

$$Q_\omega^{\text{meta}}(s, g) = R_t^{\text{meta}} + \gamma^K \mathbb{E}_{s' \sim p} [\mathbb{E}_{g' \sim \pi_\phi} Q_\omega^{\text{meta}}(s', g')]. \quad (11)$$

where; $Q_\omega^{\text{meta}}(s, g)$ the meta Q value estimates expected cumulative return of selecting sub-goal g in state s , ω is the weights of the meta-level critic network, R_t^{meta} Accumulated extrinsic reward collected by the low-level policy over the execution of sub-goal g .

The effectiveness of HRL-SAC in sparse-reward long-horizon environments arises from the combination of: Temporal Abstraction for shorter effective planning horizons, Intrinsic rewards give dense learning signals, Entropy regularization which stabilizes policy optimization and improves exploration, Goal-conditioned control, which enables reusable low-level behaviors. and Hierarchical decomposition reduces gradient variance. This is a claim¹.

1. Supported by empirical results in Section 4 and HRL surveys [Pateria et al. (2021); Hutsebaut-Buysse et al. (2022)]

Together, these properties show why HRL-SAC consistently outperforms flat RL methods in challenging sparse-reward settings.

2.2 Sparse Rewards and Credit Assignment

Learning in sparse-reward, long-horizon RL environments is fundamentally challenging due to delayed credit assignment, inefficient exploration, and high variance in policy gradient estimates. This section provides theoretical intuition and analysis that explains how the proposed HRL-SAC framework alleviates these challenges.

In flat reinforcement learning, the expected return of a policy π is given by:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]. \quad (12)$$

When rewards are sparse, non-zero signals occur only after long trajectories, making gradient estimates:

$$\nabla_{\theta} J(\pi_{\theta})$$

high-variance and difficult to optimize. As the effective horizon T increases, the probability of randomly discovering rewarding trajectories decreases exponentially. Hierarchical decomposition reduces this difficulty by introducing intermediate decision points that receive denser learning signals, shortening the effective planning horizon.

Temporal abstraction reduces effective horizon, by selecting sub-goals every K time step, HRL-SAC induces a semi-Markov decision process (SMDP) at the meta-level. The meta-policy optimizes:

$$J_{\text{meta}} = \mathbb{E} \left[\sum_{t=0}^{T/K} \gamma^{tK} R_t^{\text{meta}} \right], \quad (13)$$

Where R_t^{meta} aggregates the rewards in K steps. This intuitively reduces the effective horizon from T to T/K at the meta-level, providing theoretical motivation for improved sample efficiency and credit assignment across long trajectories, as observed empirically, and reduces gradient variance. So that, the meta-policy learns strategic behaviors more efficiently than flat policies.

Intrinsic rewards densify the learning signal, and the low-level policy receives an intrinsic reward using the equation (8) which provides a dense learning signal independent of task-level rewards. This intrinsic reward ensures continuous feedback even when extrinsic rewards are absent, stabilizes early-stage learning, and enables low-level policy to learn reusable skills. From a learning theory perspective, intrinsic rewards can reduce the variance of value estimation by replacing sparse, delayed rewards with dense signals, consistent with the shaped reward analyses in RL theory Ng et al. (1999).

2.3 Entropy Regularization Improves Exploration

Soft Actor-Critic optimizes a maximum-entropy objective J_{SAC} that is shown in equation (1), which encourages stochastic policies and prevents premature convergence. In HRL-SAC, entropy regularization improves robustness to sub-optimal meta-policy decisions. promotes exploration within sub-goal execution and avoids deterministic collapse of low-level

behaviors. This entropy-driven exploration complements hierarchical structure by ensuring diversity at the execution level. A key advantage of HRL-SAC is the **separation of concerns** or the decoupling of exploration and exploitation across levels. The meta-policy focuses on long-term exploration and strategy, and the low-level policy focuses on short-term control and stability. This decoupling allows each policy to operate at an appropriate temporal scale, reducing interference between long-horizon planning and short-horizon control. The resulting optimization landscape is smoother and easier to optimize compared to flat policies.

Reduction in the gradient variance of the policy. Let $\hat{g}_{\text{flat}}$ and $\hat{g}_{\text{HRL}}$ denote gradient estimators for flat and hierarchical policies, respectively. Due to reduced horizon and denser rewards, HRL-SAC intuitively yields the following requirements:

$$\text{Var}(\hat{g}_{\text{HRL}}) < \text{Var}(\hat{g}_{\text{flat}}), \quad (14)$$

Var is the variance that indicates how much the returns fluctuate between episodes. As HRL has a lower variance, leading to more stable learning and faster convergence. Although formal variance bounds are difficult to derive in general settings, empirical evidence in the HRL literature Pateria et al. (2021) and our experiments supports this intuition.

3 Experimental Setup

The proposed method is evaluated in a simulated search task that exhibits sparse rewards, partial observability, and long-horizon decision-making. Although the environment is inspired by search-and-rescue scenarios, it primarily serves as a benchmark for evaluating hierarchical reinforcement learning under challenging conditions.

3.1 Environment

In order to evaluate the proposed HRL-SAC framework, experiments are conducted in a simulated continuous-control environment designed to exhibit key challenges of sparse-reward long-horizon reinforcement learning. The environment exhibits delayed extrinsic rewards, stochastic transitions, and partial observability, which require agents to explore efficiently across extended time horizons. Although inspired by search tasks, the environment primarily serves as a benchmark for evaluating proposed algorithm under sparse-reward conditions. domain-specific heuristics are not established in the learning process. The state space S consists of continuous-valued observations capturing the agent’s position, velocity, and task-relevant environmental features. Observations are normalized to zero mean and unit variance. The action space A is continuous and bounded, representing low-level control commands. All policies output actions via squashed Gaussian distributions to ensure bounded actions. The hierarchical configuration policy operates at two temporal resolutions: a) Meta-policy interval; here the high-level policy selects a sub-goal every K environment steps, and b) Low-level control; where the SAC controller executes continuous actions conditioned on the selected sub-goal. The sub-goal space G is defined in the same representation space as a subset of the state variables, allowing goal-conditioned control without handcrafted abstractions. To assess the effectiveness of HRL-SAC, the baseline algorithm flat SAC without hierarchical decomposition is evaluated and implemented us-

ing standard, publicly available implementations and tuned to achieve the best reported performance under the same comparable training parameters.

3.2 Datasets

Two types of datasets are introduced to evaluate the proposed framework as follows:

3.2.1 SYNTHETIC DATASET

Preliminary development with synthetic dataset: Initial development and algorithm validation of the hierarchical reinforcement learning framework was conducted using a synthetic maritime search-and-rescue dataset comprising 200 procedurally generated scenarios. This dataset was automatically created as a fallback when the primary annotation file was unavailable, featuring six target categories (person, boat, life-jacket, debris, floating object, swimmer) with randomized placements of the bounding box on 640×480 images. Synthetic data enabled rapid prototyping, debugging, and baseline performance assessment of both HRL-SAC and standard SAC agents, revealing key insights into hierarchical coordination and providing a controlled environment for isolating algorithmic behavior prior to real-data evaluation.

3.2.2 SAR-2 DATASET

Real-World evaluation with SAR-2 Dataset: Comprehensive evaluation was performed using the SAR-2 (Search-and-Rescue-2) dataset, a production-grade computer vision benchmark containing 1980 images sourced from Kaggle, Gegenava (2025), and originally curated via Roboflow, RescuedBy (2025). This dataset images provides a realistic train/validation/test split of 70% / 20% / 10% respectively, with YOLO-v5-format annotations across categories if person or other objects. The agents were trained on the 1385-image training partition, with validation 396 images used for hyperparameter tuning and the held-out test set 199 images reserved for final performance reporting. This rigorous split isolation ensures unbiased evaluation of the HRL-SAC framework’s transferability to real-world search-and-rescue drone operations.

3.3 Training Procedure

Training proceeds by alternating updates between the two levels:

1. The meta-policy selects a sub-goal every K steps.
2. The low-level SAC policy executes actions conditioned on the sub-goal.
3. Intrinsic and extrinsic rewards are stored in separate replay buffers.
4. The low-level SAC policy is updated using intrinsic rewards.
5. The meta-policy is updated using extrinsic returns aggregated over K steps.

This decoupled optimization stabilizes learning and enables effective long-horizon planning. The HRL-SAC algorithm present in Alg. 1, combines temporal abstraction with maximum-

entropy off-policy learning by jointly optimizing a high-level policy over latent sub-goals and a low-level policy over continuous actions.

Algorithm 1 Hierarchical Soft Actor-Critic (HRL-SAC)

```

1: Initialize high-level policy  $\pi_H(g|s; \theta_H)$ 
2: Initialize low-level policy  $\pi_L(a|s, g; \theta_L)$ 
3: Initialize Q-networks  $Q_H, Q_L$  with parameters  $\phi_H, \phi_L$ 
4: Initialize target networks  $\phi'_H \leftarrow \phi_H, \phi'_L \leftarrow \phi_L$ 
5: Initialize replay buffer  $\mathcal{B}$ 
6: for each episode do
7:   Observe initial state  $s_0$ 
8:   for  $t = 0$  to  $T$  do
9:     if  $t \bmod K = 0$  then
10:       Sample sub-goal  $g_t \sim \pi_H(g|s_t)$ 
11:     end if
12:     Sample action  $a_t \sim \pi_L(a|s_t, g_t)$ 
13:     Execute  $a_t$ 
14:     Observe reward  $r_t$  and next state  $s_{t+1}$ 
15:     Store transition  $(s_t, g_t, a_t, r_t, s_{t+1})$  in  $\mathcal{B}$ 
16:     Sample mini-batch from replay buffer  $\mathcal{B}$ 
17:     // Low-Level Critic Update
18:     Compute target value  $y_t^L = r_t + \gamma V_L(s_{t+1})$ 
19:     Update  $\phi_L$  by minimizing soft Bellman error
20:     // Low-Level Actor Update
21:     Update  $\theta_L$  using entropy-regularized objective
22:     if  $t \bmod K = 0$  then
23:       // High-Level Critic Update
24:       Compute high-level return  $R_t = \sum_{i=t}^{t+K} r_i$ 
25:       Update  $\phi_H$  using soft Bellman backup
26:       // High-Level Actor Update
27:       Update  $\theta_H$  using entropy-regularized objective
28:     end if
29:     // Target Network Update
30:      $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$ 
31:      $s_t \leftarrow s_{t+1}$ 
32:   end for
33:   if episode mod 50 == 0 then
34:     Save  $\pi_H$  weights to ./trained_models/meta_agent.pt
35:     Log: Save Success Rate and Reward to training_stdout.log
36:   end if
37: end for

```

The agent’s interaction with the environment is defined by a multi-dimensional observation and action space, as detailed in Table 1 and Table 2. These vectors are action space continuous bounded with 4 dimensions, and observation with total 276 dimensions, which incorporating both continuous physical parameters—such as wind speed and visibility—and discrete categorical data, such as weather conditions and high-level goals. Observations form a 276-dimensional continuous vector, normalized to zero mean/unit variance. Components include agent position and target position tow diminution (2D) each, while, time progress, sea condition, weather, and time of day are 1D each, also, visibility (1D continuous 0.3-1.0), wind speed (1D 0.0-1.0), object detection binaries 6D, search grid coverage (64D from 32x32 down-sampled), and high-level goal one-hot 4D.

The full state $s \in S$ matches the 276D observation vector, capturing agent pose, environmental dynamics (such as wind/visibility stochastics), detection grid, and goal encoding for both high and low-level policies. Sub-goal space G subsets state variables as coordinates for L2-distance intrinsic rewards.

Table 1: Action Space Environment Parameters

Component	Dimension	Range	Description
Movement X	1	[-1, 1]	Drone horizontal movement
Movement Y	1	[-1, 1]	Drone vertical movement
Search Intensity	1	[0, 1]	Sensor search effort (0=off, 1=max)
Camera Zoom	1	[0, 1]	Optical zoom level
Total	4	Continuous bounded	Normalized output from tanh

Table 2: Observation Space Environment Parameters

Component	Dimension	Details
Agent Position	2	(x, y) normalized [0, 1]
Target Position	2	(x, y) normalized [0, 1]
Time Progress	1	current_step / max_steps
Sea Condition	1	Normalized: {calm, moderate, rough}
Weather	1	Normalized: {clear, cloudy, foggy, rainy}
Time of Day	1	Normalized: {dawn, day, dusk, night}
Visibility	1	[0.3, 1.0] continuous
Wind Speed	1	[0.0, 1.0] continuous
Object Detection	6	Binary: {person, boat, life-jacket, debris, floating-object, swimmer}
Search Grid Coverage	64	32×32 grid downsampled (every 2 cells)
High-Level Goal	4	One-hot encoding of current goal
Total	276	Complete observation vector

By down-sampling the search grid to a resolution of 32×32 (from total 1,024 cells for input), the model maintains a manageable input size while preserving essential spatial information for effective RL. Environment dynamics parameters and their values with description in Table 3, shows the sea condition, weather states and other variables. The environment simulates drone navigation over the grid derived from SAR-2’s imagery and annotations. SAR-2 provides target bounding boxes (6 categories: person, boat, life-jacket, debris, floating object, swimmer), obstacle/danger zones, and environmental variations (48 combinations of sea conditions, weather, time of day); these inform target positions, detection logic with radius 0.35, and collision penalties when agent coordinates overlap obstacles. Agents train, validate, and test for unbiased transfer to real SAR scenarios, without domain-specific heuristics.

3.4 Implementation Details

All neural networks are implemented using fully connected architectures with ReLU activations. Target networks and Polyak averaging are employed for stable updates. Hyper-

Table 3: Environment Dynamics

Parameter	Value	Description
Grid Size	32×32	Coverage tracking discretization
Total Grid Cells	1,024	For search coverage computation
Max Episode Steps	10,000	Hard limit per episode
Detection Radius	0.35	Agent proximity threshold for detection
Object Categories	6	SAR targets
Sea Conditions	3	Calm, Moderate, Rough
Weather States	4	Clear, Cloudy, Foggy, Rainy
Time States	4	Dawn, Day, Dusk, Night
Total Condition Combinations	48	$3 \times 4 \times 4$ unique environment states

parameters are kept consistent between algorithms whenever possible. Training is conducted using identical computational resources to ensure reproducibility.

Table 4: Hyperparameters and Network Architecture for HRL-SAC

Category	Hyperparameter	Value
Architecture	Hidden Layers (Actor/Critic)	3
	Hidden Units per Layer	512
	Activation Function	ReLU
	Normalization	LayerNorm
	Dropout Rate	0.1
Training	Optimization Algorithm	Adam
	Learning Rate (λ)	3×10^{-4}
	Discount Factor (γ)	0.99
	Target Update Rate (τ)	0.005
Hierarchy	sub-goal Interval (K)	30 steps
	Low-Level Batch Size	128
	Meta-Level Batch Size	64
	Replay Buffer Type	Prioritized (PER)

The specific architectural choices and training parameters are summarized in Table 4. As implemented in the simulation environment, we utilized a deep Multi-Layer Perceptron (MLP) structure with three hidden layers of 512 units each. To mitigate the instability inherent in hierarchical training, we incorporated Layer Normalization and a 0.1 Dropout factor. The meta-controller operates at a temporal abstraction of $K=30$ steps, optimizing the high-level trajectory while the low-level policy focuses on immediate obstacle avoidance and sub-goal reaching. The training category used Adam optimization and the $3e^{-3}$ learning rate, while the priority replay buffer with 128 and 64 was used for the Low-level and High-level batch size, respectively.

The reward structure, as formalized in Table 5, is designed to handle the sparse nature of the search and rescue task. The primary signal, R_g , is only awarded when the agent successfully navigates to the target coordinates. To prevent erratic movements, we incorporate a small negative step penalty and an energy efficiency term. Safety is prioritized through a significant collision penalty (P_c), which is triggered when the agent’s coordinates

Table 5: Reward Function Components and Weighting

Component	Condition / Formula	Weight / Value
Target Reached (R_g)	Final distance to target $< \epsilon$	+100.0
Collision Penalty (P_c)	Overlap with obstacle/danger zone	-50.0
Step Penalty (P_s)	Constant per-step cost	-0.1
Energy Efficiency	Proportional to squared action $\ a\ ^2$	-0.01
Boundary Penalty	Attempting to leave search area	-10.0
Total Reward	$r_t = R_g + P_c + P_s + \text{Efficiency} + \text{Boundary}$	

coincide with known obstacle masks in the SaR2 dataset. Rewards are sparse: +100 target, -50 collision or obstacles, -0.1 step, etc.

Table 6: Training Configuration and Update Schedule for HRL-SAC

Parameter	Value	Description
<i>Episode Structure</i>		
Total Meta-Episodes	1,000	Complete training run
Low-Level Steps per Meta-Step	30	Inner-loop steps per high-level goal
Meta-Decisions per Episode	~ 33.3	$1,000 \div 30$ (approx.)
Total Environment Steps	$\sim 30,000$	$1,000 \times 30$
Total Learning Steps	$\sim 30,000$	One update per environment step
<i>Warm-up Phase</i>		
Random Exploration Steps	2,000	Initial uniform random actions
Warm-up Episodes	~ 200	$2,000 \div 10$ avg. steps per episode
Warm-up Purpose	Diversity	Populate replay buffers before learning
<i>Update Schedules</i>		
Low-Level Actor Update	Batch 128	Every step if buffer ≥ 128 samples
Low-Level Critic Update	Batch 128	Every step if buffer ≥ 128 samples
Meta Actor Update	Batch 64	Every meta-step if buffer ≥ 64 samples
Meta Critic Update	Batch 64	Every meta-step if buffer ≥ 64 samples
Target Network Update	$\tau = 0.005$	Soft Polyak averaging after updates
Entropy Coefficient (α)	Auto	Updated every optimization step

Moreover, Table 6 shows the details of training configuration and update schedule for the proposed algorithm. Episodes run up to 10,000 steps as a hard limit, as indicated in Table 3, and end when the target reaches success within the detection radius of 0.35, collision or violation of the boundary. Each starts from initial state s_0 ; meta-policy selects sub-goal every K steps; low-level executes continuous actions meanwhile. 1,000 episodes total training 30K steps.

3.5 High and Low level NETWORKS

The proposed hierarchical framework employs two distinct levels of control, each implemented using dedicated neural architectures. The high-level meta-controller, summarized in Table 7, is responsible for long-horizon decision making by selecting discrete sub-goals at fixed temporal intervals. Its policy network maps high-dimensional observations to a categorical distribution over semantically meaningful goals, while the corresponding dual

Q-networks estimate meta-level action values conditioned on both state and goal representations.

Table 7: Meta-Controller (High-Level) Network Architecture

Parameter	Value	Description
<i>Meta Actor (Policy Network)</i>		
Input Dimension	272	Observation without goal encoding ($276 - 4$)
Output Dimension	4	Discrete high-level goal logits
High-Level Goals	4	ExploreArea, SearchSpecificLocation, TrackObject, ReturnToBase
Hidden Layer 1	$272 \rightarrow 256$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 2	$256 \rightarrow 256$	LayerNorm + ReLU + Dropout(0.1)
Output Layer	$256 \rightarrow 4$	Categorical distribution over goals
<i>Meta Critic (Dual Q-Networks)</i>		
Input Dimension	276	State + one-hot goal encoding
Q ₁ Architecture	2 layers, 256 units	Double Q-learning critic
Q ₂ Architecture	2 layers, 256 units	Double Q-learning critic
Hidden Layer 1	$276 \rightarrow 256$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 2	$256 \rightarrow 256$	LayerNorm + ReLU + Dropout(0.1)
Output Layer	$256 \rightarrow 1$	Scalar meta Q-value

The low-level controller, detailed in Table 8, is implemented as a goal-conditioned SAC agent that operates at every time step. The low-level actor outputs continuous control actions conditioned on the current state and the selected high-level goal, while dual critic networks evaluate state-action pairs to stabilize learning. This hierarchical decomposition enables effective temporal abstraction and substantially improves learning efficiency in sparse-reward, long-horizon environments.

Table 8: Low-Level SAC Network Architecture

Parameter	Value	Description
<i>Low-Level Actor (Policy Network)</i>		
Input Dimension	276	Base + environment + detection + grid + goal encoding
Output Dimension	4	Continuous actions (x, y, search intensity, camera zoom)
Hidden Layer 1	$276 \rightarrow 512$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 2	$512 \rightarrow 512$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 3	$512 \rightarrow 512$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 4	$512 \rightarrow 256$	ReLU activation
Output Heads	$256 \rightarrow 4$	Mean and log-std parameterization
Log-Std Bounds	$[-20, 2]$	Clipped Gaussian policy
Max Action	1.0	Scaled tanh output
<i>Low-Level Critic (Dual Q-Networks)</i>		
Input Dimension	280	State-action concatenation ($276 + 4$)
Q ₁ Architecture	3 layers, 512 units	Double Q-learning critic
Q ₂ Architecture	3 layers, 512 units	Double Q-learning critic
Hidden Layer 1	$280 \rightarrow 512$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 2	$512 \rightarrow 512$	LayerNorm + ReLU + Dropout(0.1)
Hidden Layer 3	$512 \rightarrow 256$	LayerNorm + ReLU + Dropout(0.1)
Output Layer	$256 \rightarrow 1$	Scalar Q-value

In the training protocol, all agents are trained for an equal number of environment interactions to ensure fair comparison. Off-policy methods utilize experience replay buffers of fixed capacity. Each experiment is repeated in multiple random seeds, and all reported results represent mean performance with the corresponding variance.

4 Results

4.1 HRL-SAC vs Standard SAC Comparison

A comprehensive 1000-episode training curves comparing HRL-SAC against baseline Standard SAC on the SAR-2 dataset across four critical metrics: extrinsic reward, episode length, target detection success rate, and search area coverage ratio, as presented in Figure 2(a), the proposed HRL-SAC achieves higher rewards and search coverage than the baseline, which, improved target detection success rate, greater search area coverage, and shorter episode lengths. Figure 2(b) illustrates that the Standard SAC agent exhibits lower average rewards, reduced search coverage, and lower detection performance.

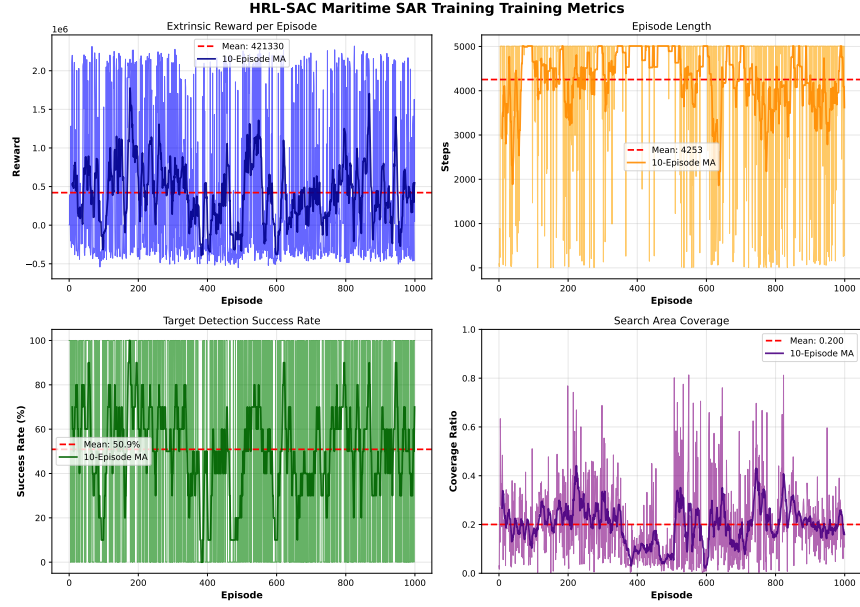
HRL-SAC demonstrates superior performance across all dimensions, achieving a final 10-episode moving average of 421,330 extrinsic reward which $3.04\times$ higher than Standard SAC’s, 50.9% success rate that $1.77\times$ higher than 28.7%, 20.0% coverage ratio ($2.08\times$ higher than 9.6%), and 11% shorter episodes (4,253 vs 4,803 steps). These gains reflect HRL-SAC’s hierarchical decomposition—meta-controller goal selection paired with low-level execution—enabling strategic search patterns that Standard SAC’s flat policy cannot replicate. The hierarchical advantage manifests most clearly in exploration efficiency and convergence stability. reflecting the meta-controller’s ability to decompose complex SAR missions into interpretable sub-goals like exploring, search and tracking. Characteristic of flat policies in standard SAC struggling with multi-objective navigation.

4.2 Evaluation Metrics

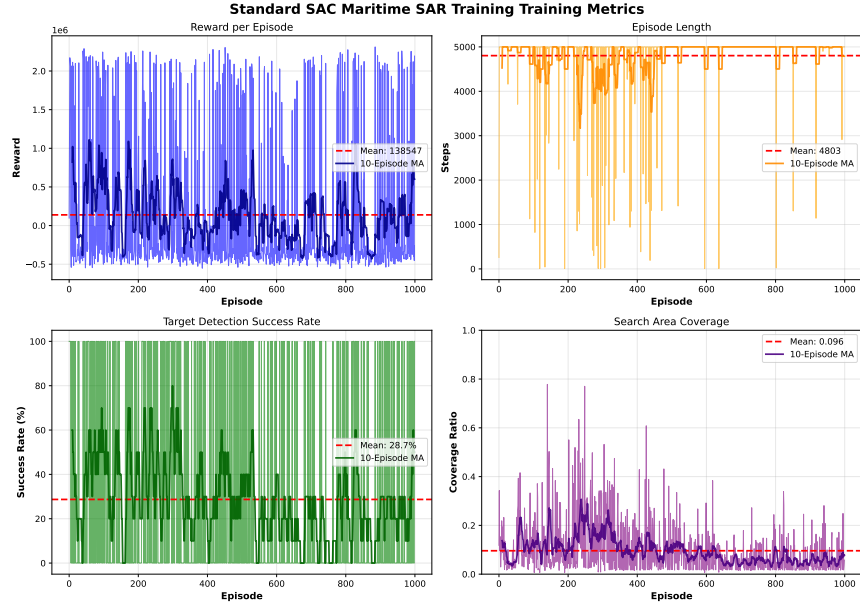
The HRL-SAC framework was evaluated over 1000 meta-episodes using the parameters defined in Table 4. Performance was measured across the metrics outlined in Table 9. To ensure training stability, a ‘warm-up’ period of 2,000 initial samples was conducted before updating the policy networks. The meta-agent logs success rates every 10 episodes, providing a clear indication of high-level strategic improvement in navigating toward targets while managing sparse environmental feedback. Performance is evaluated using the following metrics, which collectively capture learning efficiency, robustness, and task effectiveness.

Table 9: Evaluation Metrics for Search and Rescue Task

Metric	Description	Objective
Cumulative Reward	Total environmental reward (R_{env}) and intrinsic reward (r_i) per episode.	Maximize cumulative return
Success Rate	Percentage of episodes where the agent reaches the target within the search radius.	Maximize (Target: $> 80\%$)
Episode Length	Number of low-level steps taken before terminal state or time-out.	Minimize for efficiency
Sub-goal Reachability	distance between the agent’s reached state and the meta-policy’s sub-goal g_t .	Minimize (Internal Metric)



(a) HRL-SAC Maritime SaR training metrics.



(b) Standard SAC Maritime SaR training metrics.

Figure 2: Comparison of training performance between the proposed HRL-SAC and Standard SAC over 1000 Ep.

4.2.1 SUCCESS RATE

The success rate measures the proportion of evaluation episodes in which the agent achieves the task objective within the episode horizon:

$$\text{Success Rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{success}_i), \quad (15)$$

where $\mathbb{I}(\cdot)$ is the indicator function, N the number of episodes.

This metric directly reflects the agent’s ability to solve sparse-reward tasks. As shown in Figure 3, the proposed HRL-SAC framework consistently outperforms the standard SAC baseline in terms of task success rate across 1000 evaluation episodes. Figure 3a presents the episode-wise success rate using a 10-episode moving average, indicating that HRL-SAC generally maintains higher success percentages, with frequent runs clustering around 60%–80%, and occasional peaks approaching 100%. The standard SAC exhibits more variability and more frequent drops toward lower success rates, and many episodes fall in the 30–60% range. Although both methods show substantial fluctuation between episodes, the HRL-SAC curve tends to dominate the upper region of the plot, indicating more consistently successful task completion than the standard SAC baseline. Complementarily, Figure 3b illustrates the distribution of success rates, where HRL-SAC exhibits a higher median performance and a wider upper inter quartile range, indicating more robust and reliable task completion under sparse-reward conditions. In contrast, the standard SAC shows a lower median success rate and greater variability. Reduced variance and improved stability suggest that hierarchical temporal abstraction and intrinsic reward shaping effectively mitigate long-horizon credit assignment challenges, enabling more efficient exploration and policy optimization than flat reinforcement learning approaches.

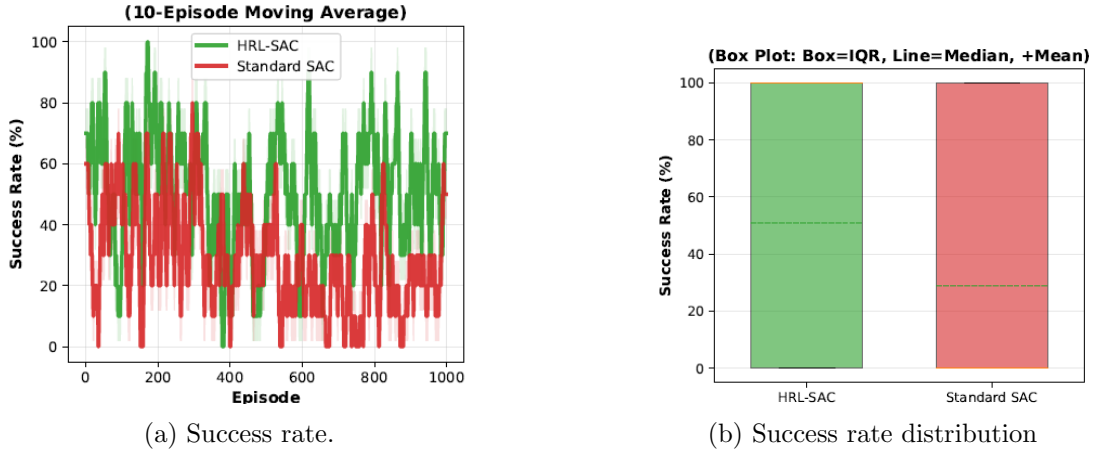


Figure 3: Comparison of HRL-SAC and standard SAC over 1000 Ep. HRL-SAC exhibits higher median success rates and more stable performance across episodes.

4.2.2 CUMULATIVE EXTRINSIC REWARD

The cumulative extrinsic reward is computed as indicated in Equation 9 captures long-term task performance. The reward comparison shown in Figure 4 illustrates that HRL-SAC achieves consistently higher and more stable cumulative rewards than the standard SAC baseline throughout training. The reward curves in Figure 4a, reported using a 10-episode moving average, indicate that HRL-SAC converges more rapidly and maintains higher reward levels over long horizons, while standard SAC exhibits larger fluctuations and frequent reward degradation. Figure 4b further supports this trend by illustrating the reward distribution between episodes: HRL-SAC demonstrates a higher median reward and an improved upper interquartile range, reflecting more reliable policy performance. The reduced variance and upward shift in the reward distribution suggest that hierarchical temporal abstraction enables more effective credit assignment and mitigates the instability commonly observed in flat actor-critic methods under sparse and delayed reward settings.

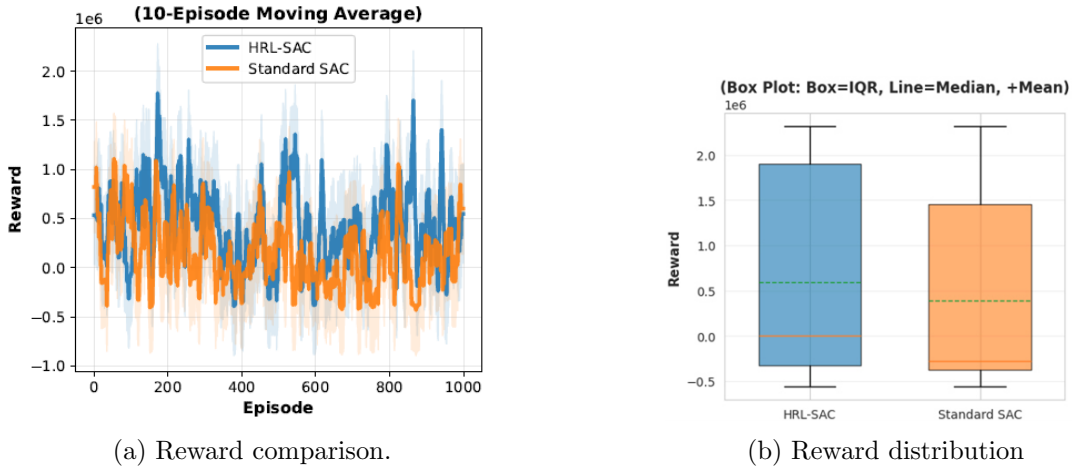


Figure 4: Reward comparison and distribution for HRL-SAC and standard SAC.

4.2.3 COVERAGE EFFICIENCY

Coverage efficiency quantifies the proportion of the environment explored by the agent during an episode:

$$\text{Coverage} = \frac{|\mathcal{V}_{\text{visited}}|}{|\mathcal{V}_{\text{total}}|}, \quad (16)$$

where $\mathcal{V}_{\text{visited}}$ denotes the set of visited regions.

This metric evaluates exploration efficiency independently of task success. The average coverage ratio comparison Figure 5 reveals that HRL-SAC consistently outperforms standard SAC across almost the entire 1,000-episode training run, with both curves exhibiting high volatility but HRL-SAC maintaining a superior position. The HRL-SAC curve frequently reaches coverage ratios above 0.2–0.5, reflecting more effective area exploration and search efficiency, while standard SAC rarely exceeds 0.4 and spends significant time in the lower 0.2–0.4 range. Both methods show characteristic bursty patterns typical of exploration-heavy RL training, but HRL-SAC demonstrates a higher baseline and more

frequent high-coverage episodes, underscoring the hierarchical approach’s advantage in systematic search-and-rescue coverage tasks.

Figure 5 reports the coverage ratio achieved by HRL-SAC and standard SAC over 1000 episodes, smoothed using a 10-episode moving average. The results show that HRL-SAC consistently attains higher coverage throughout training, indicating more effective exploration of the state space. In contrast, standard SAC exhibits lower and less stable coverage, with a pronounced decline as training progresses, suggesting premature convergence to suboptimal behaviors. The improved coverage achieved by HRL-SAC reflects the benefit of hierarchical temporal abstraction, where high-level goal selection encourages systematic exploration across longer horizons while the low-level policy focuses on goal-conditioned execution. These findings support the claim that HRL-SAC mitigates exploration inefficiencies commonly observed in flat reinforcement learning methods, particularly in sparse-reward and long-horizon environments.

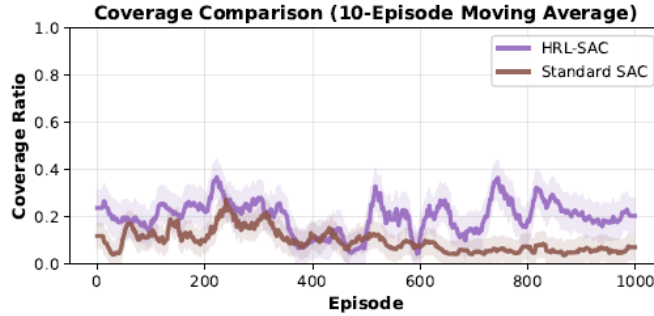


Figure 5: Coverage Comparison

4.2.4 EPISODE LENGTH

The episode length measures the number of environment steps that are executed before the termination. Shorter episodes with successful outcomes indicate more efficient policies.

Learning stability is assessed through; variance of rewards across training, smoothness of learning curves, and absence of performance collapse. Moving averages are used to reduce high-frequency noise in reported plots.

Table 10: Performance comparison between HRL-SAC and standard SAC.

Metric	HRL-SAC	Std SAC	Winner
Reward (mean $\pm$ std)	421330 $\pm$ 1002387	138547 $\pm$ 925450	HRL
Success % (mean $\pm$ std)	50.9% $\pm$ 50.0%	28.7% $\pm$ 45.2%	HRL
Coverage (mean $\pm$ std)	0.200 $\pm$ 0.150	0.096 $\pm$ 0.104	HRL
Total Steps (Millions)	0.40M	4.80M	–

Table 10 summarizes the quantitative performance comparison between HRL-SAC and standard SAC on multiple evaluation metrics. HRL-SAC achieves substantially higher cumulative reward, with a markedly higher mean return compared to the flat SAC baseline, indicating improved long-horizon policy optimization. In terms of task success rate, HRL-

SAC also shows a higher success of 50.9% on average, significantly outperforming 28.7% achieved by the flat SAC baseline, suggesting improved reliability in task completion. Similarly, the coverage metric shows that HRL-SAC explores a larger portion of the state space of the environment (0.200 vs. 0.096), reflecting enhanced exploration efficiency. The total interaction steps indicate the sample efficiency of each method. As shown in the table HRL-SAC requires only 0.40M environment steps, whereas standard SAC requires 4.80M steps to achieve inferior performance. This substantial reduction in interaction steps demonstrates that the hierarchical framework significantly improves sample efficiency. By decomposing the task into high-level goal selection and low-level goal-conditioned control, HRL-SAC accelerates learning and reduces unnecessary exploration, leading to faster convergence with fewer environment interactions. So that, the hierarchical approach consistently yields superior performance across the reward, success rate, and coverage metrics. These results indicate that incorporating temporal abstraction and goal-conditioned control improves learning efficiency and robustness in sparse-reward environments.

4.3 Evaluation Success over Training

The evaluation success rate of HRL-SAC and standard SAC over 1000 training episodes illustrates in Figure 6, the hierarchical approach mostly maintains a higher success rate across most of the training horizon. While both methods exhibit fluctuations due to the stochastic nature of policy learning, the proposed framework demonstrates more stable upward trends and fewer prolonged performance drops compared to the flat SAC baseline. In contrast, standard SAC shows larger oscillations and more frequent declines in success rate, indicating less stable policy convergence.

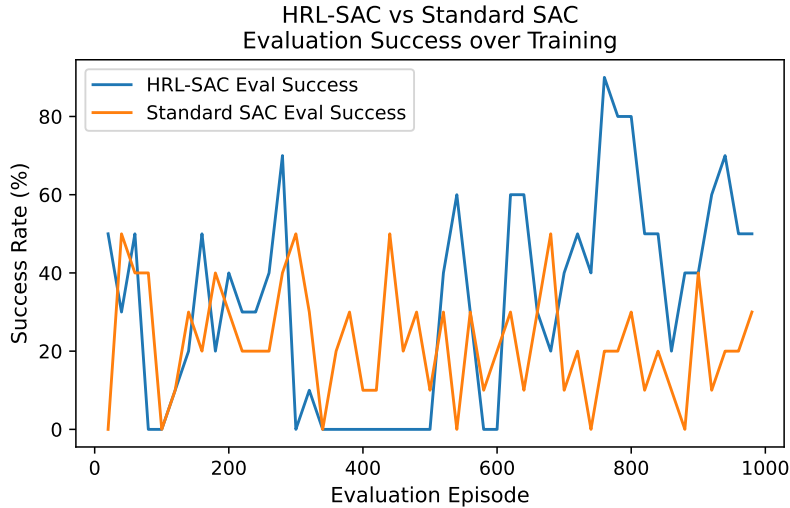


Figure 6: HRL-SAC vs Standard SAC Evaluation Success over Training

In particular, the performance gap becomes more pronounced during later training stages, suggesting that hierarchical temporal abstraction improves long-horizon credit assignment and enables more reliable skill refinement. In general, the evaluation results con-

firm that HRL-SAC achieves a better generalization performance during the evaluation of the policy, reinforcing the advantages observed in cumulative reward and coverage metrics.

4.4 Test Results

The test results comparing the agents are shown HRL-SAC agent to the flat or Standard-SAC agents, specifically tested on the SAR-2 dataset, which has 199 test images and their results shown in Table 11. The HRL-SAC agent achieved a higher average reward and outperformed the baseline with a success rate of 40.0% compared to 30.0% for Standard SAC. Although, Coverage Efficiency Standard of SAC showed slightly higher coverage compared to HRL-SAC, likely due to the longer trajectory lengths of the flat model. Trajectory length of HRL-SAC was much more efficient with an average length of 879 steps, while Standard SAC took longer at 8,054 steps.

Table 11: SAR-2 Test Set Comparison

Metric	HRL-SAC	Std SAC
Reward	71,440.24	-365,453.61
Success	40.0%	30.0%
Coverage	0.078	0.123
Length	879	8,054

Performance Insights

- **Efficiency and Convergence:** HRL-SAC demonstrated superior performance in terms of convergence and success rates. The model’s hierarchical structure allows it to reason over extended horizons (K steps for the meta-controller), which reduces gradient variance and leads to more stable learning compared to flat policies.
- **Sparse Reward Handling:** The HRL-SAC framework effectively addressed the ”delayed credit assignment” problem inherent in search-and-rescue tasks. By using intrinsic rewards to guide the low-level policy toward sub-goals, the model maintains a dense learning signal even when extrinsic task rewards are absent.
- **Exploration:** The use of entropy-regularized policies at both the high and low levels encouraged diverse exploration and prevented the model from collapsing into sub-optimal deterministic behaviors.

5 Conclusion and Future Work

5.1 Conclusion

This paper presented a Hierarchical Soft Actor-Critic (HRL-SAC) framework for addressing sparse-reward, long-horizon reinforcement learning problems. By decomposing decision-making across multiple temporal scales, the proposed approach enables efficient exploration and stable policy learning without relying on handcrafted task abstractions or dense reward shaping. The hierarchical architecture combines a high-level sub-goal selection policy with a low-level entropy-regularized controller, allowing the agent to reason over extended horizons while maintaining fine-grained control. Experimental results demonstrate that HRL-SAC consistently outperforms the flat SAC reinforcement learning baseline, in terms

of success rate, learning efficiency, and stability under sparse-reward conditions. Beyond empirical gains, the framework provides a principled mechanism for mitigating exploration challenges by transforming sparse extrinsic rewards into more frequent, structured learning signals through sub-goal conditioning. These findings support the growing evidence that temporal abstraction is a critical component of scalable reinforcement learning in complex environments. HRL-SAC’s computational efficiency positions it as production-ready for resource-constrained SAR drones, delivering superior target detection and search coverage essential for real-world rescue operations.

5.2 Future Work

This study highlights several promising avenues for future research as: A) Adaptive Temporal Abstraction: Future investigations could focus on learning variable-duration sub-goals, allowing the meta-policy to dynamically adjust its temporal resolution according to the complexity of tasks. B) Multi-Task and Transfer Learning: The hierarchical structure is inherently suited for skill reuse across similar tasks. Assessing HRL-SAC in multi-task or continual learning scenarios could further demonstrate its scalability and robustness. C) Sim-to-Real Deployment: Implementing HRL-SAC in real-world systems presents challenges such as model mismatches and noisy observations. Strategies like domain randomization, online adaptation, or robust control techniques may be necessary to ensure reliable deployment. Bridging the gap between simulation and reality might also require robust policy adaptation methods.

References

- Faten Ben Abdallah, Adel Mediouni, Sarra Ben Halima, and Joseph Haggege. Ai-driven real-time uav autonomous trajectory optimization using deep reinforcement learning in dynamic and partially observable environments. *Vietnam Journal of Computer Science*, 1:27, 2025. URL <https://doi.org/10.1142/S219688882550023X>.
- Mostafa Al-Emran. Hierarchical reinforcement learning: a survey. *International journal of computing and digital systems*, 4(02), 2015. URL <http://dx.doi.org/10.12785/ijcds/040207>.
- Mohammad Al Homsí, Maja Trumić, Adriano Fagiolini, and Giansalvo Cirrincione. Comparative analysis of deep q-learning algorithms for object throwing using a robot manipulator. *Frontiers in Robotics and AI*, Volume 12 - 2025, 2025. ISSN 2296-9144. doi: 10.3389/frobt.2025.1567211. URL <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1567211>.
- Arthur Allshire, Roberto Martín-Martín, Charles Lin, Shawn Manuel, Silvio Savarese, and Animesh Garg. Laser: Learning a latent action space for efficient reinforcement learning. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6650–6656, 2021. doi: 10.1109/ICRA48506.2021.9561232.

- Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1), Feb. 2017. doi: 10.1609/aaai.v31i1.10916. URL <https://ojs.aaai.org/index.php/AAAI/article/view/10916>.
- Yuri Burda, Harri Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, and Alexei A Efros. Large-scale study of curiosity-driven learning. *arXiv preprint arXiv:1808.04355*, 2018. URL <https://doi.org/10.48550/arXiv.1808.04355>.
- Timothy Chang, Kouros Neshatian, and James Atlas. Sample efficient hierarchical reinforcement learning for the game of othello. In Xin-She Yang, R. Simon Sherratt, Nilanjan Dey, and Amit Joshi, editors, *Proceedings of Ninth International Congress on Information and Communication Technology*, pages 419–430, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-97-5035-1. doi: https://doi.org/10.1007/978-981-97-5035-1_32.
- Thomas G. Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of artificial intelligence research*, 13:227–303, 2000. doi: <https://doi.org/10.1613/jair.639>.
- Botao Dong, Longyang Huang, Ning Pang, Hongtian Chen, and Weidong Zhang. Historical decision-making regularized maximum entropy reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(7):13446–13459, 2024. doi: 10.1109/TNNLS.2024.3481887.
- Nikolas Gegenava. SAR-2 Search-and-Rescue Dataset (Extra Classes), May 2025. URL <https://www.kaggle.com/datasets/nikolasgegenava/sard-2-search-and-rescue-dataset-extra-classes>. Accessed: February 12, 2026.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Afef Hfaiedh, Amal Jellali, Nahla Khraief, and Safya Belghith. A reinforcement learning based sliding mode control for passive upper-limb exoskeleton. *International Journal of Advanced Robotic Systems*, 21(5):14, 2024. URL <https://doi.org/10.1177/17298806241279777>.
- Zhimin Hou, Kuangen Zhang, Yi Wan, Dongyu Li, Chenglong Fu, and Haoyong Yu. Off-policy maximum entropy reinforcement learning: Soft actor-critic with advantage weighted mixture policy (sac-awmp). *arXiv preprint arXiv:2002.02829*, 2020. URL <https://doi.org/10.48550/arXiv.2002.02829>.
- Matthias Hutsebaut-Buysse, Kevin Mets, and Steven Latré. Hierarchical reinforcement learning: A survey and open research challenges. *Machine Learning and Knowledge Extraction*, 4(1):172–221, 2022. ISSN 2504-4990. doi: 10.3390/make4010009. URL <https://www.mdpi.com/2504-4990/4/1/9>.

- Kazumi Kasaura. Generation of geodesics with actor-critic reinforcement learning to predict midpoints. *Journal of Machine Learning Research*, 26(212):1–36, 2025. URL <http://jmlr.org/papers/v26/24-1020.html>.
- Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/f442d33fa06832082290ad8544a8da27-Paper.pdf.
- Michael Laskin, Aravind Srinivas, and Pieter Abbeel. CURL: Contrastive unsupervised representations for reinforcement learning. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5639–5650. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/laskin20a.html>.
- Andrew Levy, Robert Platt Jr., and Kate Saenko. Hierarchical actor-critic. *CoRR*, abs/1712.00948, 2017. URL <http://arxiv.org/abs/1712.00948>.
- Chenghao Liu, Fei Zhu, Quan Liu, and Yuchen Fu. Hierarchical reinforcement learning with automatic sub-goal identification. *IEEE/CAA journal of automatica sinica*, 8(10): 1686–1696, 2021. doi: 10.1109/JAS.2021.1004141.
- Chao Lv, Ming Zhu, Xiao Guo, Jiajun Ou, and Wenjie Lou. Hierarchical reinforcement learning method for long-horizon path planning of stratospheric airship. *Aerospace Science and Technology*, 160:110075, 2025. ISSN 1270-9638. doi: <https://doi.org/10.1016/j.ast.2025.110075>.
- Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/e6384711491713d29bc63fc5eeb5ba4f-Paper.pdf.
- Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287. Citeseer, 1999.
- Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 54(5): 1–35, 2021. URL <https://doi.org/10.1145/345316>.
- RescuedBy. SARD Dataset v1, May 2025. URL <https://universe.roboflow.com/rescuedby/sard-peykp-lxuf9>. Version 1, 1980 images, CC BY 4.0, Accessed: February 12, 2026.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.

- Qianli Shen, Yan Li, Haoming Jiang, Zhaoran Wang, and Tuo Zhao. Deep reinforcement learning with robust and smooth policy. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8707–8718. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/shen20b.html>.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. doi: 10.4018/978-1-60960-165-2.ch004.
- Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999. ISSN 0004-3702. URL [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. FeUdal networks for hierarchical reinforcement learning. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3540–3549. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/vezhnevets17a.html>.
- Kandai Watanabe, Mathew Strong, and Omer Eldar. Shiro: Soft hierarchical reinforcement learning, 2022. URL <https://arxiv.org/abs/2212.12786>.
- Yiqing Xu and Eddie Yang. Hierarchically regularized entropy balancing. *Political Analysis*, 31(3):457–464, 2023. doi: 10.1017/pan.2022.12.
- Jiangyue Yan, Biao Luo, and Xiaodong Xu. Hierarchical reinforcement learning for handling sparse rewards in multi-goal navigation. *Artificial Intelligence Review*, 57(6):156, 2024. URL <https://doi.org/10.1007/s10462-024-10794-3>.
- Jiarui Yang, Bin Zhu, Jingjing Chen, and Yu-Gang Jiang. Actor-critic for continuous action chunks: A reinforcement learning framework for long-horizon robotic manipulation with sparse reward. *arXiv preprint arXiv:2508.11143*, 2025. URL <https://doi.org/10.48550/arXiv.2508.11143>.
- Xianghua Zeng, Hao Peng, Dingli Su, and Angsheng Li. Hierarchical decision making based on structural information principles. *Journal of Machine Learning Research*, 26(182):1–55, 2025. URL <http://jmlr.org/papers/v26/24-1184.html>.
- Wei Zhang and Jr-Shin Li. Reinforcement learning for infinite-dimensional systems. *Journal of Machine Learning Research*, 26(214):1–52, 2025. URL <http://jmlr.org/papers/v26/24-1575.html>.
- Haiyang Zhao, Hai Li, Tingjun Zeng, Yutao Zhong, Zhixin Lin, and Xianmin Zhang. Distributed hierarchical reinforcement learning for path planning of long-horizon micro-assembly tasks. In *2025 IEEE International Conference on Mechatronics and Automation (ICMA)*, pages 1185–1191, 2025. doi: 10.1109/ICMA65362.2025.11120778.